

Lezione 6 - Rivisitare la ricerca binaria

INTRODUZIONE

◆ Idea

In questa lezione analizziamo la **ricerca binaria** dal punto di vista della memoria.

- 👉 non ci interessa solo l'algoritmo
- 👉 ma **come interagisce con la cache**

◆ Obiettivo

Vogliamo capire:

- come la ricerca binaria accede alla memoria
- quali pattern di accesso utilizza
- se è possibile migliorare le prestazioni

◆ Osservazione importante

La ricerca binaria è un caso interessante perché combina due comportamenti diversi:

- accessi con buona **località temporale**
- accessi con buona **località spaziale**

◆ Domanda chiave

- 👉 possiamo trovare una disposizione in memoria più efficiente?

PATTERN DI ACCESSO

◆ Idea

Analizziamo come la ricerca binaria accede alla memoria.

👉 ci interessano due fattori:

- località temporale
- località spaziale

◆ Località temporale

Durante la ricerca binaria:

👉 alcuni elementi vengono visitati molto spesso

👉 in particolare:

- gli elementi “al centro” dell’array
- perché ogni ricerca passa da lì

◆ Interpretazione

Possiamo vedere la ricerca binaria come un **albero**:

- la radice è il valore centrale
- poi si scende a sinistra o destra

👉 quindi:

- i nodi vicini alla radice sono visitati più spesso
- i nodi in basso meno frequentemente

✳️ questo significa:

👉 buona **località temporale** per i valori centrali

◆ Località spaziale

Durante la ricerca:

👉 il comportamento cambia nel tempo

👉 all'inizio:

- si fanno salti grandi nell'array
- si accede a posizioni molto lontane

👉 quindi:

- bassa località spaziale

👉 verso la fine:

- l'intervallo diventa piccolo
- si accede a elementi vicini

👉 quindi:

- buona località spaziale

◆ Riassunto

La ricerca binaria ha:

- buona località temporale (valori centrali)
- località spaziale variabile:

- bassa all'inizio
- alta alla fine

DISPOSIZIONE IN MEMORIA

◆ Idea

Possiamo migliorare le prestazioni cambiando il modo in cui i dati sono disposti in memoria.

👉 l'array ordinato può essere visto come un **albero binario**

◆ Rappresentazione come albero

Consideriamo un array ordinato.

👉 possiamo interpretarlo così:

- il valore centrale → radice
- la metà sinistra → sottoalbero sinistro
- la metà destra → sottoalbero destro

👉 quindi:

- posizione centrale → nodo principale
- poi si divide ricorsivamente

✳ la ricerca binaria segue proprio questa struttura

◆ Problema

Nell'array normale:

👉 gli elementi sono in ordine lineare

👉 ma:

- i nodi “vicini” nell’albero
- NON sono vicini in memoria

👉 esempio:

- radice → centro
- figli → posizioni lontane

✳️ quindi la cache non è sfruttata bene

◆ Domanda

👉 possiamo rappresentare lo stesso albero in modo diverso?

👉 cioè:

- mantenere la struttura logica
- ma migliorare la disposizione in memoria

◆ Idea chiave

Possiamo memorizzare l’albero direttamente in un array:

👉 senza usare puntatori

👉 usando questa regola:

- nodo in posizione i
- figlio sinistro → $2i$
- figlio destro → $2i + 1$

◆ Consequenza

👉 in questa rappresentazione:

- i nodi vicini nell'albero
- sono anche più vicini in memoria

✳️ quindi:

- migliore località
- meno cache miss

📌 COSTRUZIONE DELL'ARRAY RIORDINATO

◆ Idea

Partiamo da un array ordinato e vogliamo costruire una nuova disposizione in memoria.

👉 obiettivo:

- rappresentare l'array come un albero
- migliorare la località

◆ Struttura dell'array

Useremo:

- array originale $\rightarrow a$
- nuovo array $\rightarrow t$

👉 importante:

- $t[0]$ NON viene usato
- l'indice parte da 1

◆ Strategia

Dato un intervallo:

1. prendiamo il valore centrale
2. lo mettiamo nella posizione k
3. costruiamo ricorsivamente:
 - sottoalbero sinistro $\rightarrow 2k$
 - sottoalbero destro $\rightarrow 2k + 1$

✳ in questo modo costruiamo l'albero dentro un array



Codice

```
int reorder_array(int * a, int * t, int len, int k, int i)
{
    if (k <= len)
    {
        i = reorder_array(a, t, len, 2*k, i);
        t[k] = a[i];
        i = i+1;
        i = reorder_array(a, t, len, 2*k + 1, i);
    }
    return i;
}
```

👉 parametri:

- $a \rightarrow$ array ordinato
- $t \rightarrow$ array riordinato
- $len \rightarrow$ dimensione
- $k \rightarrow$ posizione nell'albero
- $i \rightarrow$ indice nell'array originale

👉 funzionamento:

1. costruisce prima il sottoalbero sinistro
2. inserisce il valore centrale
3. costruisce il sottoalbero destro

👉 quindi:

- visita in-order
- ma scrive in posizione “albero”

◆ Risultato

👉 otteniamo un array che:

- rappresenta un albero binario
- è più efficiente per la cache

📌 RICERCA SU ARRAY RIORDINATO

◆ Idea

Ora che l'array è organizzato come un albero, possiamo fare la ricerca seguendo direttamente questa struttura.

👉 invece di lavorare su intervalli (l, r) , lavoriamo su indici dell'albero

◆ Strategia

Partiamo dalla radice:

👉 $i = 1$

Ad ogni passo:

- se il valore è quello cercato → trovato

- se la chiave è minore → vai a sinistra
- se la chiave è maggiore → vai a destra

👉 usando le regole:

- sinistra → $2*i$
- destra → $2*i + 1$

👉 si termina quando:

- $i > \text{len}$



Codice

```
int search_reordered(int * v, int len, int key)
{
    int i = 1;
    int pos = -1;
    while (i <= len)
    {
        if (v[i] == key)
        {
            pos = i;
        }
        i = 2*i + (key > v[i]);
    }
    return pos;
}
```

◆ Spiegazione

👉 iniziamo dalla radice ($i = 1$)

👉 ad ogni iterazione:

- controlliamo $v[i]$
- se è uguale $\rightarrow$ salviamo la posizione

👉 aggiornamento indice:

```
i = 2*i + (key > v[i]);
```

👉 significa:

- se $key \leq v[i] \rightarrow 2*i$ (sinistra)
- se $key > v[i] \rightarrow 2*i + 1$ (destra)

👉 quindi:

- niente `if` espliciti
- comportamento simile a branchless

◆ Vantaggi

👉 rispetto alla binary search classica:

- accessi più regolari
- migliore località in memoria

◆ Idea generale

La ricerca segue la struttura dell'albero direttamente:

👉 meno salti "casuali"

👉 accessi più prevedibili

✳️ questo migliora l'uso della cache

PREFETCH NELLA RICERCA RIORDINATA

◆ Idea

Possiamo migliorare ulteriormente le prestazioni usando il **prefetch**.

👉 cioè:

- caricare in anticipo dati in cache
- prima ancora di usarli

◆ Osservazione

Nella ricerca su albero:

👉 sappiamo già quali nodi visiteremo

👉 infatti:

- ad ogni livello accediamo a un nodo
- poi scendiamo al livello successivo

👉 quindi:

- possiamo prevedere accessi futuri

✳️ questo rende il prefetch molto efficace



Codice

```
int search_reordered_prefetch(int * v, int len, int key)
{
    int i = 1;
    int pos = -1;
    while (i <= len)
    {
        if (v[i] == key)
        {
            pos = i;
        }

        __builtin_prefetch(v + i*16);
        __builtin_prefetch(v + i*16 + 15);

        i = 2*i + (key > v[i]);
    }
    return pos;
}
```

◆ Spiegazione

👉 oltre alla ricerca normale, aggiungiamo:

```
__builtin_prefetch(...)
```

👉 questo dice alla CPU:

- “prepara questi dati in cache”

👉 in particolare:

- pre-carichiamo un blocco di elementi
- che probabilmente useremo

◆ Perché funziona

👉 nella rappresentazione ad albero:

- i nodi dello stesso livello sono “vicini” in memoria
- spesso stanno nella stessa cache line

👉 quindi:

- il prefetch carica più dati utili insieme
- riduce la latenza

◆ Idea generale

👉 combinando:

- disposizione in memoria intelligente
- prefetch

otteniamo:

- meno cache miss
- accessi più veloci

✳ massimo sfruttamento della cache

📌 CONFRONTO DELLE VERSIONI

◆ Idea

Confrontiamo le diverse versioni della ricerca binaria:

- con branch
- branchless
- branchless con prefetch
- ricerca su array riordinato
- ricerca riordinata con prefetch



Codice di test

```
float test_search(int (*f) (int*, int, int), int size, int search_size)
{
    int * v = random_vector(size);
    qsort(v, size, sizeof(int), cmpfunc);
    int * keys = random_vector(search_size);
    int * pos = (int *) malloc(search_size * sizeof(int));

    clock_t start = clock();
    for (int i = 0; i < search_size; i++)
    {
        pos[i] = f(v, size, keys[i]);
    }
    clock_t end = clock();

    float ms = (float) (end - start) / (CLOCKS_PER_SEC / 1000.0);

    free(v);
    free(keys);
    free(pos);

    return ms;
}
```

👉 questa funzione:

- esegue molte ricerche
- misura il tempo totale



Versione per array riordinato

```
float test_search_reordered(int (*f) (int*, int, int), int size, int search_size)
{
    int * v = random_vector(size);
    qsort(v, size, sizeof(int), cmpfunc);

    int * v_reorder = (int *)malloc((size + 1) * sizeof(int));
    v[0] = -1;

    reorder_array(v, v_reorder, size, 1, 0);
    free(v);

    int * keys = random_vector(search_size);
    int * pos = (int *) malloc(search_size * sizeof(int));

    clock_t start = clock();
    for (int i = 0; i < search_size; i++)
    {
        pos[i] = f(v_reorder, size, keys[i]);
    }
    clock_t end = clock();

    float ms = (float) (end - start) / (CLOCKS_PER_SEC / 1000.0);

    free(v_reorder);
    free(keys);
    free(pos);

    return ms;
}
```

👉 differenza:

- prima si riordina l'array
- poi si misura la ricerca



Programma principale

```
for (int i = exp_min; i <= exp_max; i++)
{
    float t;

    t = test_search(binary_search, 1<<i, search_size);
    t = test_search(binary_search_branchless, 1<<i, search_size);
    t = test_search(binary_search_branchless_prefetch, 1<<i, search_size);

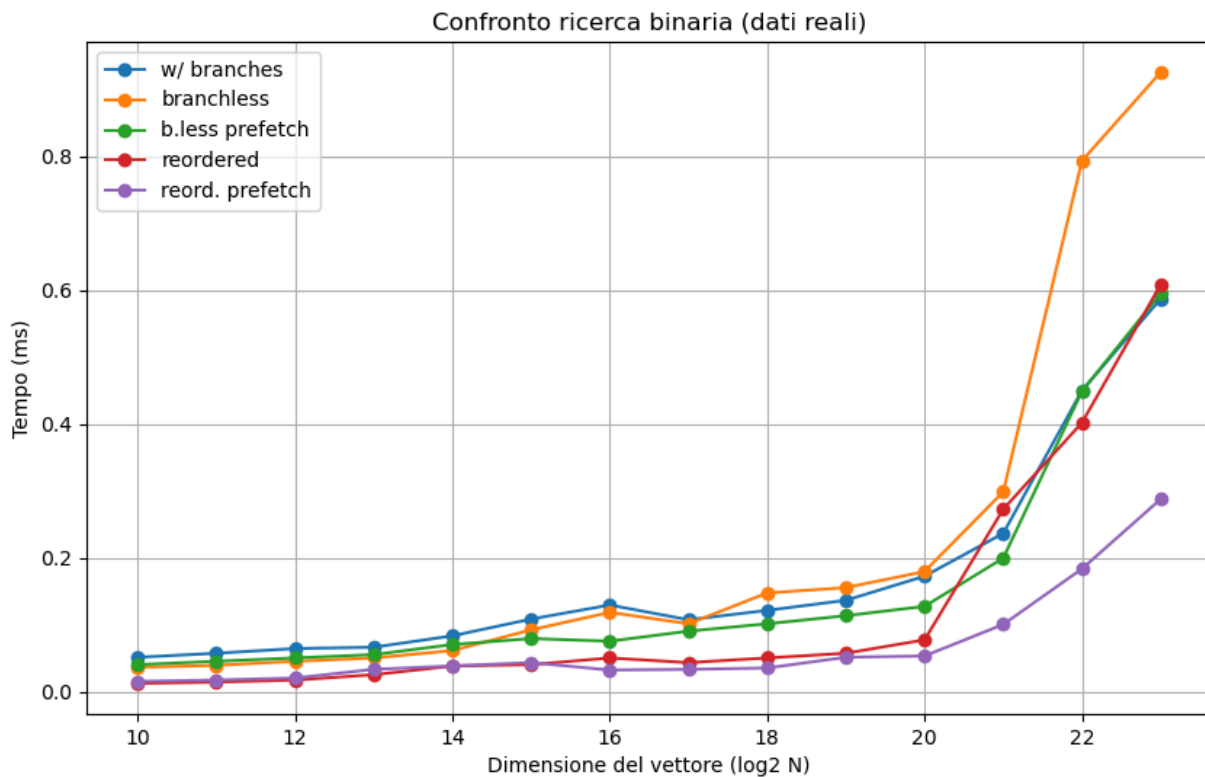
    t = test_search_reordered(search_reordered, 1<<i, search_size);
    t = test_search_reordered(search_reordered_prefetch, 1<<i, search_size);
}
```

👉 questo ciclo:

- testa array di dimensione crescente
- confronta tutte le versioni

◆ Grafico dei risultati

Il grafico seguente mostra il confronto tra le diverse versioni della ricerca binaria al variare della dimensione del vettore (dati ottenuti sperimentalmente).



👉 Osservazioni:

- per dimensioni piccole → tutte le versioni hanno prestazioni simili
- per dimensioni grandi → emergono differenze significative

👉 in particolare:

- la versione con branch peggiora
- la versione branchless può peggiorare per input grandi
- il prefetch migliora le prestazioni
- la versione riordinata migliora la località
- la versione riordinata con prefetch è la migliore

✳️ questo dimostra che la memoria e la cache influenzano fortemente le prestazioni

◆ Problema

👉 anche la versione riordinata ha un limite:

- i figli (2^k , 2^{k+1})
- per valori grandi di k

👉 quindi:

- non stanno nella stessa cache line
- la località spaziale peggiora

◆ Miglioramento

👉 possiamo usare il prefetch anche qui:

- pre-carichiamo blocchi di nodi
- riduciamo i cache miss

◆ Idea generale

Le prestazioni dipendono da:

- disposizione in memoria
- predicibilità degli accessi
- utilizzo della cache

✳ non basta l'algoritmo → conta come accediamo ai dati

📌 CONCLUSIONI

◆ Idea

Abbiamo visto come la ricerca binaria può essere migliorata considerando la memoria.

◆ Osservazione 1

👉 le versioni senza branch:

- evitano errori di predizione
- possono essere più efficienti

◆ Osservazione 2

👉 quando la dimensione dei dati cresce:

- la cache diventa il fattore principale
- la latenza della memoria domina

◆ Osservazione 3

👉 la disposizione in memoria è fondamentale:

- array standard → accessi poco locali
- array riordinato → accessi più prevedibili

◆ Osservazione 4

👉 il prefetch permette di:

- anticipare gli accessi
- ridurre i cache miss

◆ Idea chiave

👉 conoscere in anticipo gli accessi alla memoria è un vantaggio enorme

◆ Riassunto finale

Possiamo migliorare le prestazioni usando:

- codice senza branch
- migliore disposizione dei dati
- prefetch

◆ **Concetto fondamentale**

👉 le prestazioni non dipendono solo dall'algoritmo

👉 ma anche da:

- memoria
- cache
- pattern di accesso

✳️ ottimizzare la memoria è fondamentale per ottenere alte prestazioni